p0995R0
Improving atomic_flag

Published Proposal, 17 March 2018

This version:
http://wg21.link/P0995r0

Authors:
JF Bastien (Apple)
Olivier Giroux (NVIDIA)
Andrew Hunter (Google)

Audience:
SG1, LEWG

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Source:
github.com/jfbastien/papers/blob/master/source/P0995r0.bs

Abstract

atomic_flag is marginally useful. Improve it in light of the new wait / notify APIs.

Table of Contents

§ 1. Introduction

C++11 added `atomic_flag` to the language as the minimally-required class which could be used to implement `atomic<>` on hardware which seemed relevant at the time. Detailed `atomic_flag` history can be found in [N2145], [N2324], and [N2393]. The specification was quite successful at minimalism—the only member functions of `atomic_flag` are `test_and_set` and `clear`—but `atomic<>` was wildly more successful and to our knowledge has always been implemented with compiler support instead of with the very inefficient (but beautifully simple) `atomic_flag`. Our experience is that `atomic_flag`'s interface is so minimal as to be mostly useless, in particular it doesn't have a method which can load the flag's value without modifying it.

We've heard of it being used as:

- A questionable spinloop (as was originally intended);
- A "check-in" flag used to know when at least one thread has reached a program location.

The one special power `atomic_flag` has is in being the only type which is guaranteed to be lock-free, albeit a mostly powerless one.

SG1 tried to salvage `atomic_flag` in [P0514R0] by adding `set`, `test`, `wait`, `wait_until`, and `wait_for` methods but decided to leave it as-is and implement efficient waiting differently, eventually going for [P0514R3].

The time has come to thank `atomic_flag` for serving its purpose as an implementability stand-in, and help it find its true purpose. We propose:

- Adding a `test` method to it as [P0514R0] did. This could technically forbids some ancestral processors from implementing modern C++, but these platforms already don't support any C++.
- Add `atomic_flag` overloads to [P0514R3]'s waiting and notify functions.
- Add optional always-lock-free integral type aliases, which are encouraged to be sized such that waiting on them is most efficient.

This paper was written in Jacksonville and presented to SG1, which unanimously forwarded the paper to LEWG. LEWG looked at the paper and took the following poll:

	SF	***F***	***N***	***A***	***SA***
Make the type aliases non-optional.	*1*	*4*	*4*	*2*	*2*

The types were made optional in case an architecture, such as PA-RISC, cannot support always-lock-free integral types because no compare-and-exchange instruction is available. There was no consensus for making aliases required, though concern was expressed that LEWG doesn't usually make functionality optional. In the C++ standard library optionality is present as follows:

- `intN_t` / `uintN_t` are mandated by C, "if an implementation provides integer types with widths of 8, 16, 32, or 64 bits, no padding bits, and (for the signed types) that have a two's complement representation";
- `abs` and `div` overloads "if and only if the type `intmax_t` designates an extended integer type";
- The library `allocator_traits` template has optional requirements.

There was also discussion about ABI breakage to `atomic_flag`. An argument was made that `atomic_flag` should also be sized such that waiting on them is most efficient (which would be an ABI breakage), and if that breakage doesn't occur then adding wait / notify overloads is actively misleading. LEWG want SG1 to reconsider whether the overloads should be provided.

§ 2. Wording

Under Header `<atomic>` synopsis [**atomics.syn**] edit as follows:

```
// 32.3, type aliases

// ...
```

```
using atomic_signed_lock_free   = see below; // optional
using atomic_unsigned_lock_free = see below; // optional
```

```
// 32.8, flag type and operations
struct atomic_flag;
```

```
bool atomic_flag_test(volatile atomic_flag*) noexcept;
bool atomic_flag_test(atomic_flag*) noexcept;
bool atomic_flag_test_explicit(volatile atomic_flag*, memory_order) noex
bool atomic_flag_test_explicit(atomic_flag*, memory_order) noexcept;
```

```
bool atomic_flag_test_and_set(volatile atomic_flag*) noexcept;
bool atomic_flag_test_and_set(atomic_flag*) noexcept;
bool atomic_flag_test_and_set_explicit(volatile atomic_flag*, memory_ord
void atomic_flag_clear(volatile atomic_flag*) noexcept;
void atomic_flag_clear(atomic_flag*) noexcept;
void atomic_flag_clear_explicit(volatile atomic_flag*, memory_order) noe
void atomic_flag_clear_explicit(atomic_flag*, memory_order) noexcept;
#define ATOMIC_FLAG_INIT see below
```

```
// 32.10, waiting and notifying functions
template <class T>
  void atomic_notify_one(const volatile atomic<T>*);
template <class T>
  void atomic_notify_one(const atomic<T>*);
```

```
void atomic_notify_one(const volatile atomic_flag*);
void atomic_notify_one(const atomic_flag*);
```

```
template <class T>
  void atomic_notify_all(const volatile atomic<T>*);
template <class T>
  void atomic_notify_all(const atomic<T>*);
```

```
void atomic_notify_all(const volatile atomic_flag*);
void atomic_notify_all(const atomic_flag*);
```

```
template <class T>
  void atomic_wait(const volatile atomic<T>*,
                   typename atomic<T>::value_type);
template <class T>
  void atomic_wait(const atomic<T>*, typename atomic<T>::value_type);
```

```
void atomic_wait(const volatile atomic_flag*, bool);
void atomic_wait(const atomic_flag*, bool);
```

```
template <class T>
  void atomic_wait_explicit(const volatile atomic<T>*,
                            typename atomic<T>::value_type,
                            memory_order);
template <class T>
  void atomic_wait_explicit(const atomic<T>*,
                            typename atomic<T>::value_type, memory_order
```

```
void atomic_wait_explicit(const volatile atomic_flag*, bool, memory_orde
void atomic_wait_explicit(const atomic_flag*, bool, memory_order);
```

In Atomic operations library [**atomics**], under Type aliases [**atomics.alias**], edit as follows:

> The type aliases `atomic_intN_t`, `atomic_uintN_t`, `atomic_intptr_t`, and `atomic_uintptr_t` are defined if and only if `intN_t`, `uintN_t`, `intptr_t`, and `uintptr_t` are defined, respectively.
>
> The type aliases `atomic_signed_lock_free` and `atomic_unsigned_lock_free` are defined to be specializations of `atomic` whose template arguments are integral types, respectively signed and unsigned, other than `bool`. `is_always_lock_free` shall be `true` for `atomic_signed_lock_free` and `atomic_unsigned_lock_free`. These aliases are optional. If an implementation provides a integral specialization of `atomic` other than `bool` for which `is_always_lock_free` is `true`, it shall define the aliases to such a type. Otherwise, they shall not be defined. An implementation should choose the integral specialization of `atomic` for which the waiting and notifying functions are most efficient.

In Atomic operations library [**atomics**], under Flag type and operations [**atomics.flag**], edit as follows:

```
namespace std {
  struct atomic_flag {
```

```
bool test(memory_order = memory_order_seq_cst) volatile noexcept;
bool test(memory_order = memory_order_seq_cst) noexcept;
```

```
  bool test_and_set(memory_order = memory_order_seq_cst) volatile noexce
  bool test_and_set(memory_order = memory_order_seq_cst) noexcept;
  void clear(memory_order = memory_order_seq_cst) volatile noexcept;
  void clear(memory_order = memory_order_seq_cst) noexcept;
  atomic_flag() noexcept = default;
  atomic_flag(const atomic_flag&) = delete;
  atomic_flag& operator=(const atomic_flag&) = delete;
  atomic_flag& operator=(const atomic_flag&) volatile = delete;
};
```

```
bool atomic_flag_test(volatile atomic_flag*) noexcept;
bool atomic_flag_test(atomic_flag*) noexcept;
bool atomic_flag_test_explicit(volatile atomic_flag*, memory_order) noex
bool atomic_flag_test_explicit(atomic_flag*, memory_order) noexcept;
```

```
  bool atomic_flag_test_and_set(volatile atomic_flag*) noexcept;
  bool atomic_flag_test_and_set(atomic_flag*) noexcept;
  bool atomic_flag_test_and_set_explicit(volatile atomic_flag*, memory_o
  bool atomic_flag_test_and_set_explicit(atomic_flag*, memory_order) noe
  void atomic_flag_clear(volatile atomic_flag*) noexcept;
  void atomic_flag_clear(atomic_flag*) noexcept;
  void atomic_flag_clear_explicit(volatile atomic_flag*, memory_order) n
  void atomic_flag_clear_explicit(atomic_flag*, memory_order) noexcept;
  #define ATOMIC_FLAG_INIT see below
}
```

The `atomic_flag` type provides the classic test-and-set functionality. It has two states, set and clear.

Operations on an object of type `atomic_flag` shall be lock-free. [*Note:* Hence the operations should also be address-free. —*end note*]

The `atomic_flag` type is a standard-layout struct. It has a trivial default constructor and a trivial destructor.

The macro `ATOMIC_FLAG_INIT` shall be defined in such a way that it can be used to initialize an object of type `atomic_flag` to the clear state. The macro can be used in the form:

```
atomic_flag guard = ATOMIC_FLAG_INIT;
```

It is unspecified whether the macro can be used in other initialization contexts. For a complete static-duration object, that initialization shall be static. Unless initialized with `ATOMIC_FLAG_INIT`, it is unspecified whether an `atomic_flag` object has an initial state of set or clear.

```
bool atomic_flag_test(volatile atomic_flag* object) noexcept;
bool atomic_flag_test(atomic_flag* object) noexcept;
bool atomic_flag_test_explicit(volatile atomic_flag* object, memory_orde
bool atomic_flag_test_explicit(atomic_flag* object, memory_order order)
bool atomic_flag::test(memory_order order = memory_order_seq_cst) volati
bool atomic_flag::test(memory_order order = memory_order_seq_cst) noexce
```

Requires: The `order` argument shall not be `memory_order_release` nor `memory_order_acq_rel`.

Effects: Memory is affected according to the value of `order`.

Returns: Atomically returns the value pointed to by `object` or `this`.

```
bool atomic_flag_test_and_set(volatile atomic_flag* object) noexcept;
bool atomic_flag_test_and_set(atomic_flag* object) noexcept;
bool atomic_flag_test_and_set_explicit(volatile atomic_flag* object, mem
bool atomic_flag_test_and_set_explicit(atomic_flag* object, memory_order
bool atomic_flag::test_and_set(memory_order order = memory_order_seq_cst
bool atomic_flag::test_and_set(memory_order order = memory_order_seq_cst
```

Effects: Atomically sets the value pointed to by `object` or by `this` to `true`. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations (4.7).

Returns: Atomically, the value of the object immediately before the effects.

```
void atomic_flag_clear(volatile atomic_flag* object) noexcept;
void atomic_flag_clear(atomic_flag* object) noexcept;
void atomic_flag_clear_explicit(volatile atomic_flag* object, memory_ord
void atomic_flag_clear_explicit(atomic_flag* object, memory_order order)
void atomic_flag::clear(memory_order order = memory_order_seq_cst) volat
void atomic_flag::clear(memory_order order = memory_order_seq_cst) noexc
```

Requires: The `order` argument shall not be `memory_order_consume`, `memory_order_acquire`, nor `memory_order_acq_rel`.

Effects: Atomically sets the value pointed to by `object` or by `this` to `false`. Memory is affected according to the value of `order`.

In Atomic operations library [**atomics**], under Waiting and notifying functions [**atomics.wait**], edit as follows:

The functions in this subclause provide a mechanism to wait for the value of an atomic object to change, more efficiently than can be achieved with polling. Waiting functions in this facility may block until they are unblocked by notifying functions, according to each function's effects. [*Note:* Programs are not guaranteed to observe transient atomic values, an issue known as the A-B-A problem, resulting in continued blocking if a condition is only temporarily met. – *End Note.*]

The functions `atomic_wait` and `atomic_wait_explicit` are waiting functions. The functions `atomic_notify_one` and `atomic_notify_all` are notifying functions.

```
template <class T>
  void atomic_notify_one(const volatile atomic<T>* object);
template <class T>
  void atomic_notify_one(const atomic<T>* object);
```

```
void atomic_notify_one(const volatile atomic_flag* object);
void atomic_notify_one(const atomic_flag* object);
```

Effects: unblocks up to execution of a waiting function that blocked after observing the result of an atomic operation X, if there exists another atomic operation Y, such that X precedes Y in the modification order of `*object`, and Y happens-before this call.

```
template <class T>
  void atomic_notify_all(const volatile atomic<T>* object);
template <class T>
  void atomic_notify_all(const atomic<T>* object);
```

```
void atomic_notify_all(const volatile atomic_flag* object);
void atomic_notify_all(const atomic_flag* object);
```

Effects: unblocks each execution of a waiting function that blocked after observing the result of an atomic operation X, if there exists another atomic operation Y, such that X precedes Y in the modification order of `*object`, and Y happens-before this call.

```
template <class T>
  void atomic_wait_explicit(const volatile atomic<T>* object,
                            typename atomic<T>::value_type old,
                            memory_order order);
template <class T>
  void atomic_wait_explicit(const atomic<T>* object,
                            typename atomic<T>::value_type old,
                            memory_order order);
```

Requires: The order argument shall not be `memory_order_release` nor `memory_order_acq_rel`.

Effects: Repeatedly performs the following steps, in order:

1. Evaluates `object->load(order) != old` then, if the result is `true`, returns.
2. Blocks until an implementation-defined condition has been met. [*Note:* Consequently, it may unblock for reasons other than a call to a notifying function. - *end note*]

```
void atomic_wait_explicit(const volatile atomic_flag* object, bool old,
void atomic_wait_explicit(const atomic_flag* object, bool old, memory_or
```

Effects: Repeatedly performs the following steps, in order:

1. Evaluates `object->test(order) != old` then, if the result is `true`, returns.
2. Blocks until an implementation-defined condition has been met. [*Note:* Consequently, it may unblock for reasons other than a call to a notifying function. - *end note*]

```
template <class T>
  void atomic_wait(const volatile atomic<T>* object,
                   typename atomic<T>::value_type old);
template <class T>
  void atomic_wait(const atomic<T>* object,
                   typename atomic<T>::value_type old);
```

```
void atomic_wait(const volatile atomic_flag* object, bool old);
void atomic_wait(const atomic_flag* object, bool old);
```

Effects: Equivalent to: `atomic_wait_explicit(object, old, memory_order_seq_cst);`

Two feature test macros should be added:

- `__cpp_lib_atomic_flag_test` implies the `test` methods for `atomic_flag` and free functions, as well as the notify and wait overloads for `atomic_flag`, are available.
- `__cpp_lib_atomic_lock_free_type_aliases` implies `atomic_signed_lock_free` and `atomic_unsigned_lock_free` types are defined.

§ References

§ Informative References

[N2145]
H.-J. Boehm, L. Crowl. C++ Atomic Types and Operations. 12 January 2007. URL: https://wg21.link/n2145

[N2324]
H.-J. Boehm, L. Crowl. C++ Atomic Types and Operations. 24 June 2007. URL: https://wg21.link/n2324

[N2393]
H.-J. Boehm, L. Crowl. C++ Atomic Types and Operations. 10 September 2007. URL: https://wg21.link/n2393

[P0514R0]
Olivier Giroux. Enhancing std::atomic_flag for waiting. 15 November 2016. URL: https://wg21.link/p0514r0

[P0514R3]
Olivier Giroux. Efficient concurrent waiting for C++20. URL: https://wg21.link/p0514r3